

RAPPORT EXPLICATIF — PAS À PAS

Système de détection & monitoring de menaces

Tout le projet expliqué de A à Z : les choix, les pourquoi, les vraies attaques simulées, les vrais bugs corrigés, et le lien avec chaque ligne de ton CV.

Elasticsearch · Kibana · Prometheus · Grafana · MITRE ATT&CK

Sommaire

0. Comment lire ce rapport
1. Pourquoi ce projet est excellent pour un portfolio
2. Vue d'ensemble de l'architecture
3. Les technologies : quoi, pourquoi, alternatives
4. MITRE ATT&CK : le langage commun de la détection
5. Collecter les logs : de la source à Elasticsearch
6. Les trois règles de détection, expliquées en détail
7. Simuler de vraies attaques, de façon responsable
8. L'observabilité : métriques vs logs, Prometheus & Grafana
9. Le déploiement réel et sa vérification
10. Les trois vrais bugs trouvés et corrigés
11. Git et GitHub
12. Le lien avec ton CV, phrase par phrase
13. Limites du projet et pistes d'amélioration réelles
14. Questions probables en entretien + éléments de réponse
15. Glossaire complet de tous les termes techniques

Comment lire ce rapport

Même principe que pour tes trois précédents rapports : chaque notion est définie la première fois qu'elle apparaît, dans un encadré coloré.

Définition

Bleu/violet : définit un terme technique simplement.

À retenir pour l'entretien

Vert : une phrase à savoir répéter à l'oral.

Piège / erreur classique

Orange : une erreur fréquente, évitée ici.

Lien avec ton CV

Rouge : relie la section à ta phrase de CV.

Vrai bug trouvé en direct

Rouge foncé : un vrai problème rencontré en construisant ce projet, diagnostiqué et corrigé — comme dans le projet DevSecOps précédent.

Ce projet est un **mini-SIEM** (Security Information and Event Management) : un système qui collecte des données de sécurité, les centralise, et applique des règles pour repérer des comportements anormaux — exactement le genre d'outil qu'utilise un centre opérationnel de sécurité (SOC) en entreprise, en beaucoup plus simple.

Pourquoi ce projet est excellent pour un portfolio

1.1 Le problème métier

Dans une vraie entreprise, des dizaines de systèmes (serveurs, applications, pare-feux) produisent des millions de lignes de logs par jour. Aucun humain ne peut les lire un par un. Un SIEM centralise ces logs et applique des règles automatiques pour faire remonter uniquement ce qui mérite l'attention d'un analyste — exactement le rôle de `detection_engine/rules.py` dans ce projet, à échelle réduite.

1.2 Ce qui rend ce projet crédible

Caractéristique	Pourquoi c'est important
De vraies attaques simulées	Pas des logs fictifs copiés d'un tutoriel : un vrai brute-force SSH, un vrai scan, de vraies tentatives d'injection, exécutés et observés.
Détections reliées à MITRE ATT&CK	Un référentiel utilisé par de vrais analystes SOC dans le monde entier — pas une classification inventée pour l'occasion.
Observabilité en plus de la sécurité	Prometheus/Grafana couvrent la partie "est-ce que le système fonctionne bien ?", complémentaire de "est-ce que quelqu'un l'attaque ?".
Des bugs réels trouvés en le testant	Un piège d'encodage qui cachait 3 attaques sur 5 — impossible à découvrir sans vraiment lancer l'attaque et vérifier le résultat.

À retenir pour l'entretien

"J'ai construit un mini-SIEM avec de vraies attaques simulées contre des cibles dédiées et isolées, pas des logs d'exemple copiés d'un tutoriel. Le moteur de détection a retrouvé les 7 attaques sur 7, chacune reliée à une technique documentée du référentiel MITRE ATT&CK."

Vue d'ensemble de l'architecture

Composant	Rôle
webapp/	Application cible (FastAPI) : génère des logs d'accès JSON et expose des métriques Prometheus
ssh_target/	Conteneur SSH volontairement vulnérable, cible du brute-force
attack_simulator/	Scripts qui déclenchent de vraies attaques contre ces cibles, sur le réseau Docker local uniquement
filebeat/	Achemine les logs des cibles vers Elasticsearch
detection_engine/	Interroge Elasticsearch et applique les règles de détection
Prometheus + Grafana	Collectent et visualisent les métriques d'infrastructure et d'application

```

flowchart LR
    BF[ssh_bruteforce.py] --> SSH[ssh-target]
    SC[web_scan.py] --> WEB[webapp]
    SSH -->|auth.log| FB[Filebeat]
    WEB -->|access.log JSON| FB
    FB --> ES[(Elasticsearch)]
    ES --> KIB[Kibana]
    ES --> RULES[detection_engine]
    RULES -->|alertes| ES
    WEB -->|/metrics| PROM[Prometheus]
    PROM --> GRAF[Grafana]
    ES --> GRAF

```

À retenir pour l'entretien

"L'architecture sépare clairement trois responsabilités : générer du signal (les cibles et les simulateurs d'attaque), le collecter et le stocker (Filebeat, Elasticsearch), et l'analyser (le moteur de détection). C'est le même découpage qu'un vrai pipeline SIEM en entreprise."

Les technologies : quoi, pourquoi, et par rapport à quelles alternatives

3.1 Elasticsearch

Définition

Elasticsearch est un moteur de recherche et d'analyse distribué, optimisé pour indexer et interroger rapidement de très grands volumes de documents (ici, des lignes de logs). C'est le "E" de la suite ELK (Elasticsearch, Logstash, Kibana).

Alternative écartée : une base de données classique (PostgreSQL, MongoDB). Elles peuvent stocker des logs, mais ne sont pas conçues pour la recherche textuelle rapide sur des millions de lignes ni pour les agrégations complexes (compter, grouper, filtrer en quelques millisecondes) — c'est justement la spécialité d'Elasticsearch.

3.2 Kibana

Définition

Kibana est l'interface de visualisation et d'exploration associée à Elasticsearch : recherche de logs, création de tableaux de bord, exploration interactive des données indexées.

3.3 Filebeat

Définition

Filebeat est un agent léger qui surveille des fichiers de logs en continu et achemine chaque nouvelle ligne vers une destination (ici, Elasticsearch), sans transformation lourde. C'est un des "Beats" de la suite Elastic, plus léger que Logstash pour un simple transport de logs.

Alternative écartée : Logstash. Logstash peut aussi transformer/parser les logs en profondeur (via des filtres "grok"), mais consomme beaucoup plus de ressources. Ce projet a fait le choix de garder le transport simple (Filebeat) et de faire le parsing léger directement en Python dans le moteur de détection — un compromis délibéré pour un projet de cette taille, documenté en Partie 5.3.

3.4 Prometheus

Définition

Prometheus est un système de collecte de métriques qui fonctionne par "scraping" : il vient lui-même interroger, à intervalle régulier, une URL `/metrics` exposée par chaque service surveillé, plutôt que d'attendre que les services lui envoient leurs données.

Alternative écartée : StatsD/Graphite. Plus anciens, moins adaptés à un environnement de conteneurs dynamiques où les cibles à surveiller changent souvent — le modèle "pull" (Prometheus va chercher les métriques) s'adapte mieux à Docker/Kubernetes que le modèle "push" (les anciens systèmes attendent que chaque service leur envoie ses données).

3.5 Grafana

Définition

Grafana est un outil de tableaux de bord qui peut interroger plusieurs sources de données différentes (Prometheus, Elasticsearch, bases de données SQL...) et les afficher côte à côte sur un même écran.

À retenir pour l'entretien

"J'ai configuré Grafana avec deux sources de données simultanément — Prometheus pour les métriques d'infrastructure, Elasticsearch pour les logs de sécurité — pour avoir un seul tableau de bord unifié plutôt que de jongler entre Grafana et Kibana."

MITRE ATT&CK : le langage commun de la détection

Définition

MITRE ATT&CK est une base de connaissances publique, maintenue par l'organisation à but non lucratif MITRE, qui catalogue les techniques réelles utilisées par les attaquants (comment ils obtiennent un accès initial, comment ils se déplacent dans un réseau, comment ils volent des données...), chacune identifiée par un code unique (ex: T1110 pour le brute-force).

4.1 Pourquoi s'appuyer sur ce référentiel plutôt qu'inventer ses propres catégories ?

Piège classique : des règles de détection "maison" sans vocabulaire commun

Si chaque équipe de sécurité invente ses propres noms pour les mêmes techniques, personne ne peut comparer ses détections avec celles d'un autre outil, d'une autre entreprise, ou d'un rapport de veille sur une nouvelle menace. MITRE ATT&CK est devenu le standard de facto : les outils de sécurité professionnels (SIEM commerciaux, EDR...) l'utilisent tous pour classer leurs alertes.

4.2 Les trois techniques couvertes dans ce projet

Code	Nom	Ce que ça veut dire concrètement
T1110	Brute Force	Deviner un mot de passe en essayant un grand nombre, automatiquement
T1595	Active Scanning	Sonder activement un système pour découvrir ses points d'entrée (pages cachées, ports ouverts)
T1190	Exploit Public-Facing Application	Exploiter une faille dans une application exposée sur Internet via son entrée utilisateur (formulaires, paramètres d'URL)

À retenir pour l'entretien

"Chaque règle de mon moteur de détection est explicitement reliée à un identifiant MITRE ATT&CK, exactement comme le ferait un vrai produit de sécurité. Ça permet à n'importe quel

analyste de comprendre immédiatement de quel type de menace il s'agit, sans avoir à lire le code de la règle."

Collecter les logs : de la source à Elasticsearch

5.1 Deux sources, deux formats très différents

Source	Format	Traitement
Logs SSH (<code>auth.log</code>)	Texte brut, format syslog classique	Filebeat l'envoie tel quel ; le parsing (extraire l'IP, l'utilisateur, succès/échec) se fait en Python via des expressions régulières
Logs de la webapp (<code>access.log</code>)	JSON structuré, une ligne = un objet JSON	Filebeat le décode directement (<code>json.keys_under_root: true</code>) : chaque champ JSON devient un champ Elasticsearch sans aucun parsing supplémentaire

Définition — Logs structurés vs non structurés

Un log structuré (comme du JSON) sépare déjà l'information en champs nommés (`client_ip` , `status` ...), directement exploitables. Un log non structuré (texte libre, comme la plupart des logs système Linux) nécessite un travail d'extraction (regex, "grok") avant de pouvoir être filtré ou agrégé proprement.

5.2 Pourquoi la webapp de ce projet écrit ses logs en JSON

Le fichier `webapp/app.py` intercepte chaque requête via un middleware et écrit une ligne JSON avec tous les champs utiles (IP, méthode, chemin, code de statut, durée). C'est un choix délibéré : dans un vrai projet professionnel, structurer ses logs dès leur émission (plutôt que d'espérer les re-parser plus tard) est une bonne pratique qui simplifie énormément toute la chaîne d'analyse en aval.

5.3 Le choix assumé de parser les logs SSH en Python plutôt qu'en Elasticsearch

Une vraie plateforme de production utiliserait un pipeline d'ingestion Elasticsearch (ou Logstash) pour transformer chaque ligne de log SSH en document structuré *au moment de l'indexation*. Ce projet fait le choix, assumé et documenté, de garder cette étape simple : Filebeat transporte le texte brut tel quel, et c'est le moteur de détection Python qui applique ses propres

expressions régulières au moment de l'analyse. Moins de composants à maintenir pour une démonstration, au prix d'un peu plus de travail par requête d'analyse.

Les trois règles de détection, expliquées en détail

6.1 Détection du brute-force SSH (T1110)

```
FAILED_SSH_RE = re.compile(r"Failed password for(?:\s+invalid user)?\s+(\S+)
from (\d+\.\d+\.\d+\.\d+)")
...
if len(attempts) >= BRUTE_FORCE_THRESHOLD: # 5 échecs
    compromised = ip in successes_by_ip
    ...
```

La règle compte les échecs d'authentification par adresse IP source. Au-delà d'un seuil (5 échecs), une alerte est levée. Point important : si la même IP a *ensuite* réussi à se connecter, la sévérité passe de "high" à "critical" et le message précise explicitement "compte probablement compromis" — la détection ne se contente pas de compter, elle interprète la séquence des évènements.

6.2 Détection du scan de reconnaissance (T1595)

La règle compte les réponses HTTP 404 (page non trouvée) par IP source. Un utilisateur normal ne génère presque jamais de 404 ; un outil de scan automatique (comme gobuster ou dirbuster, utilisés aussi bien par des attaquants que par des pentesters légitimes) en génère des dizaines en quelques secondes, en testant des chemins connus pour contenir parfois des informations sensibles (`/.env`, `/.git/config` ...).

6.3 Détection des tentatives d'injection (T1190)

Définition — Injection

Une injection consiste à glisser, dans une donnée censée être une simple valeur (un texte de recherche, un nom d'utilisateur), un fragment de code que le système va interpréter involontairement — une requête SQL cachée (SQLi), un script exécuté dans le navigateur d'une victime (XSS), ou un chemin de fichier système (traversée de répertoire).

La règle recherche des patterns connus (UNION SELECT , OR '1'='1' , <script> , ../ ...) dans les paramètres de requête, **après décodage** — voir la Partie 10.2 pour le bug réel que ça a permis de découvrir et corriger.

À retenir pour l'entretien

"Mes règles ne se contentent pas de chercher un mot-clé isolé : la règle de brute-force corrèle deux évènements dans le temps (échecs puis succès) pour distinguer une attaque en cours d'une attaque réussie, ce qui change complètement la priorité de la réponse à apporter."

Simuler de vraies attaques, de façon responsable

7.1 Le principe : isolation complète

Un principe non négociable

Toutes les attaques simulées de ce projet ciblent exclusivement des conteneurs créés spécifiquement pour cet usage (`ssh-target` , `webapp`), sur un réseau Docker privé, jamais exposés à Internet, jamais dirigés vers une machine tierce. C'est la différence fondamentale entre un test de sécurité légitime (sur son propre système, avec son consentement) et une attaque réelle contre un système qui ne vous appartient pas.

7.2 La simulation de brute-force SSH

`attack_simulator/ssh_bruteforce.py` utilise la librairie **Paramiko** (un client SSH en Python) pour essayer, un par un, une liste de mots de passe faibles courants contre le compte `admin` du conteneur `ssh-target` , volontairement configuré avec un mot de passe faible (`changeme123`) placé délibérément en fin de liste — pour que l'attaque "réussisse" au bout d'un moment, comme dans un vrai scénario.

7.3 La simulation de scan et d'injection

`attack_simulator/web_scan.py` génère trois types de trafic : du trafic légitime (pour que la détection doive vraiment distinguer le normal de l'anormal, pas juste "signaler tout"), un scan de chemins inexistants, et des tentatives d'injection avec des payloads connus. Chaque catégorie utilise une adresse IP simulée différente (via l'en-tête `X-Forwarded-For`), documentée avec des adresses de la plage RFC 5737 réservée à la documentation (jamais de vraies adresses IP).

À retenir pour l'entretien

"J'ai pris soin de générer aussi du trafic légitime en plus du trafic malveillant : un détecteur qui signale tout serait aussi inutile qu'un détecteur qui ne signale rien. Le vrai défi, c'est de distinguer les deux — ce que mes règles font en se basant sur des seuils et des patterns précis, pas sur une simple présence de trafic."

L'observabilité : métriques vs logs, Prometheus & Grafana

8.1 Logs vs métriques : deux types de données très différents

	Logs	Métriques
Nature	Évènements discrets, souvent textuels	Nombres qui évoluent dans le temps (compteurs, jauges)
Volume	Potentiellement énorme	Compact (un point de donnée par intervalle de temps)
Usage typique	"Que s'est-il passé exactement à cet instant ?"	"Comment la charge évolue-t-elle sur les dernières heures ?"
Outil dans ce projet	Elasticsearch / Kibana	Prometheus / Grafana

Les deux sont complémentaires : une métrique peut révéler *qu'un* pic de trafic anormal a eu lieu, un log permet ensuite de comprendre *pourquoi* et *qui* l'a causé.

8.2 PromQL, le langage de requête de Prometheus

```
sum by (status) (rate(http_requests_total[1m]))
```

Cette requête (utilisée dans le tableau de bord de ce projet) calcule le nombre de requêtes HTTP par seconde sur la dernière minute, regroupées par code de statut (200, 404, 500...). Le métriques elles-mêmes sont exposées automatiquement par la librairie `prometheus-fastapi-instrumentator`, intégrée à la webapp en 2 lignes de code.

8.3 Un tableau de bord Grafana unique pour deux mondes

Le dashboard provisionné dans ce projet (`grafana/dashboards/security-monitoring.json`) mélange volontairement des panels Prometheus (charge CPU, requêtes par seconde, latence) et un panel Elasticsearch (table des alertes de sécurité détectées) sur un seul écran — la démonstration concrète que Grafana peut servir de point d'entrée unique pour l'exploitation *et* la sécurité, plutôt que deux outils cloisonnés.

Le déploiement réel et sa vérification

Toute la chaîne a été exécutée pour de vrai sur cette machine (preuves complètes dans `reports/deployment_evidence.txt`) :

1. 9 conteneurs démarrés via `docker compose up` (Elasticsearch, Kibana, Filebeat, Prometheus, Grafana, node-exporter, cAdvisor, webapp, ssh-target).
2. Attaque SSH réelle exécutée : 10 mots de passe testés, échec, puis succès avec le mot de passe faible.
3. Scan et injections réels exécutés contre la webapp : 26 requêtes de scan, 5 tentatives d'injection.
4. Vérification directe dans Elasticsearch : 218 documents de logs indexés.
5. Moteur de détection exécuté : **7 alertes sur 7 attaques** retrouvées et classées MITRE ATT&CK.
6. Chaque cible Prometheus vérifiée individuellement (`up`) avant de faire confiance au tableau de bord.
7. 6 tests automatisés exécutés et passés (`pytest`).

À retenir pour l'entretien

"Je ne me suis pas contenté de vérifier que les conteneurs démarraient : j'ai vérifié que chaque maillon de la chaîne transportait réellement la donnée jusqu'au bout, en interrogeant directement Elasticsearch et Prometheus à chaque étape, plutôt que de faire confiance à l'apparence du tableau de bord."

Les trois vrais bugs trouvés et corrigés

10.1 `service rsyslog start` ne fonctionne pas dans un conteneur minimal

Le problème constaté

Le conteneur `ssh-target` plantait immédiatement avec l'erreur `rsyslog: unrecognized service`.

Cause : la commande `service` s'appuie sur des scripts d'init système absents dans une image Debian slim pensée pour les conteneurs. **Correction :** lancer le démon directement (`/usr/sbin/rsyslogd`).

10.2 `unquote()` ne décode pas les "+" — 3 injections sur 5 passaient inaperçues

Le problème constaté

Sur 5 tentatives d'injection envoyées et bien indexées dans Elasticsearch, le moteur de détection n'en repérait que 2 au premier essai.

Cause : une requête HTTP encode les espaces en `%20` ou en `+`. `urllib.parse.unquote()` ne décode que les séquences `%XX`, laissant les "+" tels quels — le payload `' OR '1'='1` devenait `' +OR+ '1'='1`, et la regex (qui attend un espace) ne correspondait plus. **Correction :** utiliser `unquote_plus()`, qui décode les deux formes à la fois. Un test de non-régression dédié vérifie maintenant explicitement ce cas.

À retenir pour l'entretien

"Ce bug est un excellent exemple de pourquoi il faut tester avec des données qui ressemblent vraiment à ce qu'un attaquant enverrait, pas des données 'propres' inventées pour le test. Un test avec une chaîne déjà décodée n'aurait jamais révélé ce problème."

10.3 Elasticsearch tué par manque de mémoire (OOM)

Le problème constaté

Au premier lancement, Elasticsearch et Kibana disparaissaient quelques secondes après leur démarrage (code de sortie 137 : SIGKILL).

Cause : la VM Docker locale n'avait que 4 Go de RAM alloués — insuffisant pour faire tourner Elasticsearch, Kibana, Prometheus, Grafana et le reste simultanément. **Correction :** augmentation de la mémoire allouée (4 → 6 Go) et démarrage progressif des services (Elasticsearch/Kibana d'abord, vérifiés stables, puis le reste), plutôt que de deviner un correctif sans diagnostic.

À retenir pour l'entretien — le fil conducteur des 3 bugs

"Les trois bugs de ce projet ont un point commun : aucun n'aurait été visible en relisant simplement le code. Il a fallu exécuter le système en conditions réelles pour les révéler. C'est exactement pour ça que je teste systématiquement mes projets en conditions réelles plutôt que de me fier à ce qui 'a l'air correct' sur le papier."

Git et GitHub

Même démarche que pour les trois autres projets : dépôt initialisé, `.gitignore` excluant l'environnement virtuel Python, commit unique avec un message décrivant précisément ce qui a été construit et vérifié, dépôt public créé et poussé via `gh repo create ... --push`.

Le lien avec ton CV, phrase par phrase

Ta ligne de CV : *"Mise en place d'une solution de collecte et d'analyse de logs pour détecter des comportements suspects."*

Mot-clé du CV	Ce que tu peux dire
"Elasticsearch"	"Centralisation de deux sources de logs hétérogènes (SSH, application web) dans Elasticsearch via Filebeat, avec vérification directe du nombre de documents indexés."
"Kibana"	"Data views configurées pour explorer les logs et les alertes de sécurité."
"Docker"	"Architecture à 9 conteneurs orchestrés par Docker Compose, incluant des cibles volontairement vulnérables isolées pour les tests d'attaque."
"détecter des comportements suspects"	"Moteur de détection avec 3 règles reliées à MITRE ATT&CK, testé contre de vraies attaques simulées : 7 attaques sur 7 détectées."
(bonus) "Prometheus / Grafana"	"Observabilité applicative et infrastructure en plus de la sécurité : un tableau de bord Grafana unique combinant métriques et logs de sécurité."

Une phrase d'ouverture possible en entretien

"C'est un mini-SIEM que j'ai construit pour comprendre comment un centre de sécurité opérationnel détecte réellement des attaques : je génère de vraies attaques contre des cibles isolées, je centralise les logs, et mon moteur de détection les retrouve et les classe selon un référentiel utilisé par de vrais analystes, MITRE ATT&CK."

Limites du projet et pistes d'amélioration réelles

Limite	Ce qu'on ferait en vrai
Parsing SSH en Python plutôt qu'en pipeline Elasticsearch	Utiliser un pipeline d'ingestion structuré (Logstash ou ingest pipeline Elasticsearch) pour un vrai volume de production.
Détection à la demande, pas en continu	Planifier l'exécution des règles toutes les X secondes (cron, ou un vrai moteur de règles comme ElastAlert).
cAdvisor sans visibilité par conteneur dans cet environnement	Fonctionnerait nativement sur un vrai cluster Linux/ Kubernetes ; documenté comme limite de l'environnement de démo (VM imbriquée).
Pas d>alerting actif (email, Slack)	Brancher les alertes indexées sur un webhook ou un connecteur Alertmanager pour une notification en temps réel.

Questions probables en entretien + éléments de réponse

Q : "Comment fonctionne votre moteur de détection ?"

R : Partie 6 — interroge Elasticsearch, applique 3 règles avec seuils et corrélation temporelle, indexe les alertes.

Q : "Pourquoi MITRE ATT&CK ?"

R : Partie 4 — vocabulaire commun de l'industrie, permet la comparabilité entre outils.

Q : "Quelle est la différence entre un log et une métrique ?"

R : Partie 8.1 — tableau comparatif, exemple concret du projet.

Q : "Racontez-moi un bug que vous avez résolu."

R : Partie 10.2 — le piège unquote/unquote_plus est un excellent exemple concret et technique.

Q : "Comment avez-vous testé vos attaques sans risque ?"

R : Partie 7.1 — isolation complète, réseau Docker privé, jamais de cible tierce.

Q : "Que feriez-vous différemment avec plus de temps ?"

R : Partie 13 — cite l>alerting actif et la détection en continu en priorité.

Glossaire complet de tous les termes techniques

Alerte

Résultat produit par une règle de détection quand ses conditions sont réunies.

Beat (Filebeat)

Agent léger de la suite Elastic qui transporte des données (logs, métriques) vers Elasticsearch.

Brute force

Technique consistant à essayer un grand nombre de mots de passe automatiquement.

cAdvisor

Outil qui expose des métriques de consommation de ressources par conteneur.

Elasticsearch

Moteur de recherche et d'analyse distribué, cœur du stockage de logs dans ce projet.

Filebeat

Agent qui surveille des fichiers de logs et achemine chaque nouvelle ligne vers une destination.

Grafana

Outil de tableaux de bord capable d'interroger plusieurs sources de données.

Injection

Insertion d'un fragment de code dans une donnée utilisateur, interprété involontairement par le système cible.

Kibana

Interface de visualisation et d'exploration associée à Elasticsearch.

Log structuré

Log dont l'information est déjà séparée en champs nommés (ex: JSON), sans besoin de parsing ultérieur.

MITRE ATT&CK

Référentiel public catalogant les techniques d'attaque réelles, chacune identifiée par un code (ex: T1110).

Node-exporter

Agent Prometheus qui expose les métriques système (CPU, mémoire, disque) d'une machine.

Prometheus

Système de collecte de métriques fonctionnant par "scraping" (interrogation périodique des cibles).

PromQL

Langage de requête utilisé pour interroger les métriques stockées dans Prometheus.

Scraping

Action pour un système de monitoring d'aller lui-même chercher des métriques auprès d'une cible, à intervalle régulier.

SIEM (Security Information and Event Management)

Système qui centralise des données de sécurité et applique des règles pour détecter des menaces.

SOC (Security Operations Center)

Équipe/centre chargé de surveiller en continu la sécurité d'un système d'information.